FROM CODE TO CONTAINER

IRON MAN LANDING PAGE

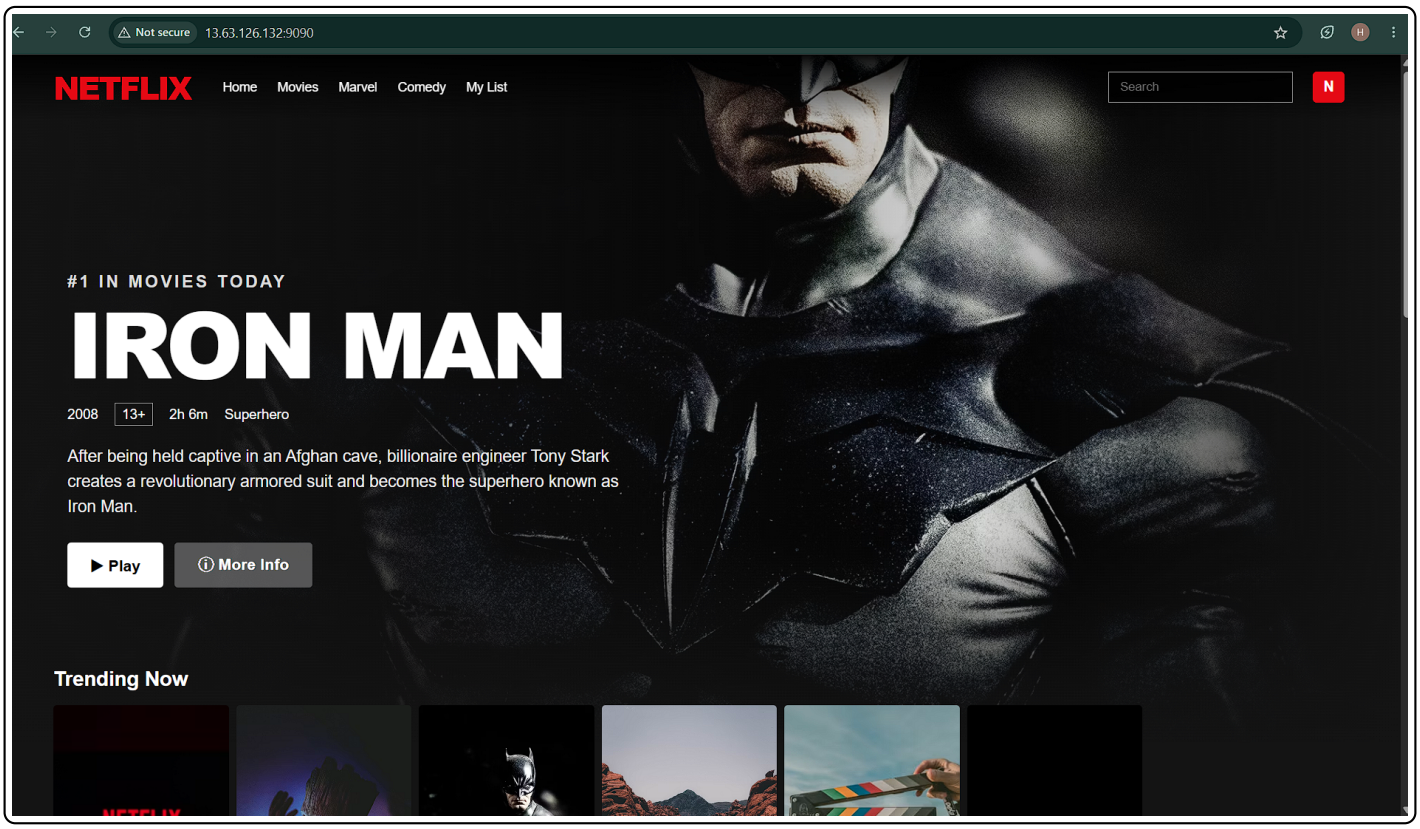
DOCKER • NGINX • AWS EC2

BASHIR SHAIKH

A practical containerization and deployment project

PROJECT STATUS: DEPLOYED & VERIFIED

Live deployment screenshot — AWS EC2 :9090



Contents

1. Project Overview

2. Architecture

3. Server Preparation

4. Docker Images & Containers

5. Nginx Deployment

6. Custom Iron Man Website

7. Dockerfile

8. Build the Image

9. Run & Verify the Container

10. Troubleshooting

11. Networking

12. Volumes

13. Docker Hub

14. Cleanup

15. Production Scenarios

16. Skills Demonstrated

17. Command Reference

18. Conclusion

This document is formatted as a professional project report suitable for sharing as a LinkedIn project/document. Sensitive host details are intentionally generalized; credentials and secrets are not included.

1. Project Overview

This project demonstrates how a custom Iron Man cinematic landing page can be containerized with Docker and served by Nginx on an AWS EC2 Linux server.

The project covers the complete learning workflow: Docker installation, official images, containers, HTML/CSS deployment, Dockerfile creation, custom image builds, port mapping, troubleshooting, Docker networking, volumes, and Docker Hub publishing.

Project branding: BASHIR SHAIKH
Application: Iron Man Landing Page
Web server: Nginx
Container image: netflix:v1
Application container: netflix-web
Published port: 9090

2. Deployment Architecture

The application is served from an Nginx container running on an AWS EC2 Docker host.

Browser
↓
EC2 public endpoint :9090
↓
Docker host
↓
netflix-web container
↓
Nginx :80
↓
/usr/share/nginx/html/index.html
↓
Iron Man Landing Page

Port mapping:

Host port **9090** → Container port **80**

The deployment was browser-tested successfully. The public IP is omitted from this shareable report to avoid unnecessarily exposing infrastructure details.

3. Server Preparation

The EC2 Linux server was prepared for Docker workloads using the following workflow:

```
sudo su
yum update -y
yum install docker -y
which docker
docker --version
systemctl start docker
systemctl status docker
systemctl enable docker
```

Purpose: install Docker, verify the CLI, start the daemon, and enable Docker at boot.

4. Docker Project Workspace

```
mkdir docker-demo
cd docker-demo
```

The workspace contains the application source file and Dockerfile.

5. Docker Images & Containers

Official images were pulled to practice Docker image and container operations.

```
docker pull nginx
docker pull ubuntu
docker images
```

Ubuntu container exercise:

```
docker run -it --name ubuntu-box ubuntu
ls
cat /etc/os-release
exit
docker ps -a
```

This demonstrates interactive container access and basic Linux inspection inside a container.

6. Nginx Deployment

```
docker run -d --name nginx-web -p 8080:80 nginx
```

The default Nginx image is exposed on EC2 port 8080 for an initial deployment test.

7. Custom Iron Man Website

The custom index.html provides a cinematic dark/red superhero interface with Iron Man content, movie rows, Marvel and comedy sections, responsive behavior, and BASHIR SHAIKH branding.

The website is copied into the standard Nginx document root during the Docker image build.

Project files:

```
docker-demo/
index.html
Dockerfile
```

8. Dockerfile

The Dockerfile packages the website with Nginx:

```
FROM nginx

LABEL maintainer="BASHIR SHAIKH"

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Using the project owner's name in the image metadata makes the deliverable consistent with the final project branding.

9. Build the Custom Image

The completed website was built into the custom image **netflix:v1**.

```
docker build -t netflix:v1 .
```

Successful image verification included:

```
docker images

REPOSITORY TAG IMAGE ID
netflix v1 fb153043488f
nginx latest f075e3f94986
```

The image build completed successfully.

10. Run the Custom Container

```
docker run -d --name netflix-web -p 9090:80 netflix:v1
docker ps
```

The verified container mapping was:

```
netflix-web
0.0.0.0:9090 -> 80/tcp
```

11. Browser Verification

The application was successfully opened in a browser using the EC2 public endpoint on port 9090.

Verification result: The custom Iron Man landing page loaded successfully through Nginx inside the Docker container.

This confirms the full path from browser → EC2 → published Docker port → Nginx → index.html.

For a public project report, the exact EC2 IP is intentionally not printed here. This keeps the report shareable without advertising a live infrastructure endpoint.

12. Troubleshooting

Core diagnostic commands:

```
docker ps
docker logs netflix-web
docker exec -it netflix-web bash
docker inspect netflix-web
docker port netflix-web
```

Inside the container, verify the website file with:

```
ls /usr/share/nginx/html
cat /usr/share/nginx/html/index.html
exit
```

13. Docker Networking

A custom network can be created to allow application containers to communicate through an isolated Docker network.

```
docker network create app-network
docker network connect app-network nginx-web
docker network connect app-network ubuntu-box
docker network connect app-network netflix-web
docker inspect app-network
```

Purpose: demonstrate custom Docker networking and container connectivity.

14. Docker Volumes

Volumes provide persistent storage outside the container's writable layer.

```
docker volume create app-data
docker run -d --name volume-nginx -v app-data:/usr/share/nginx/html nginx
```

Purpose: demonstrate persistent web-content storage.

15. Docker Hub Publishing

After logging into Docker Hub, the image can be tagged with the account namespace and pushed to a repository.

```
docker login
docker tag netflix:v1 yourdockerhubusername/netflix:v1
docker push yourdockerhubusername/netflix:v1
```

Replace *yourdockerhubusername* with the actual Docker Hub username. Never place passwords, access tokens, or secrets inside this document.

16. Cleanup

```
docker stop $(docker ps -aq)
docker rm -f $(docker ps -aq)
docker system prune -a
```

These commands are destructive. They should only be used when the containers and unused resources are no longer required.

17. Production Scenarios

Website Down: start with docker ps, then inspect logs and port mappings.

Container Error: use docker logs netflix-web and docker inspect netflix-web.

Port Issue: verify docker port netflix-web and the AWS security-group rule for the published port.

Data Loss: use Docker volumes when application data must persist beyond container replacement.

Container Access: use docker exec -it netflix-web bash for interactive inspection.

18. Skills Demonstrated

Docker installation • Docker images • Containers • Dockerfile • Nginx deployment • HTML/CSS website deployment • Port mapping • AWS EC2 deployment • Container troubleshooting • Docker networking • Docker volumes • Docker Hub workflow • Production-oriented documentation

19. Command Reference

Purpose	Command
Check Docker	docker --version
List images	docker images
Running containers	docker ps
All containers	docker ps -a
Build image	docker build -t netflix:v1 .
Run application	docker run -d --name netflix-web -p 9090:80 netflix:v1
Logs	docker logs netflix-web
Enter container	docker exec -it netflix-web bash
Inspect	docker inspect netflix-web
Port mapping	docker port netflix-web
Network	docker network create app-network
Volume	docker volume create app-data
Docker Hub	docker login

20. Conclusion

The Iron Man landing page was successfully containerized and deployed with Docker and Nginx on AWS EC2. The project demonstrates a practical container workflow from source code to image, from image to running container, and from container to browser-accessible application.

BASHIR SHAIKH

Docker • Nginx • AWS EC2 • HTML/CSS

Prepared as a professional project document for portfolio and LinkedIn sharing.